

机器人训练，到底训练什么？——再理解数据格式选型（细版）

零声教育 Darren

读这份的时机：在背 HDF5 / LeRobot / Parquet 之前，先建立「策略在学什么」。
配套文档：[面试准备 具身数据与标注.md](#)（格式、对齐、清洗、标注、ROS2 回环；数字与手算以那份中篇为准）。

一句话定位：模型侧大家都能抄开源；数据怎么喂，才决定策略能不能用。本文把「训什么 / 怎么训」讲细，再自然推出「为什么需要那些格式、怎么选」。

怎么读

目标	建议路径
建立直觉	§ 1 图1 → § 2 图2/图3 → § 10 总图
搞清张量与因果	§ 3
搞清 数据格式长什么样	§ 6（目录树 + 表结构）
搞清算法差异	§ 4（有哪些方法 + 为什么这么训）
搞清训练循环卡在哪	§ 5
搞清为何倒逼格式 / 怎么选	§ 7 → § 8
临场口述	§ 9

目录

1. [一句话：训的是策略函数 \$\pi\$](#)
2. [用抓方块建立直觉（含多频率原始数据）](#)
3. [输入 / 输出长什么样（张量、因果、归一化）](#)
4. [具体有哪些训练方法？为什么这么训？](#)
5. [训练循环细讲：从 Dataset 到 loss](#)
6. [数据格式长什么样（LeRobot / HDF5 / Parquet）](#)
7. [为什么这就倒逼这些格式](#)
8. [怎么选格式：从训练需求反推](#)
9. [面试口述与追问](#)
10. [一张图串起来](#)
11. [补充深挖：chunk / 扩散 / 增强 / 归一化](#)

1. 一句话：训的是策略函数 π

1.1 定义

训练一个策略 Policy：输入是当前及近期传感器观测，输出是机器人下一步或未来几步动作。

图 1 · 策略函数 π 在干什么



- θ ：神经网络权重，CNN / Transformer / 扩散网络等
- o_t ：观测 observation，来自传感器与状态估计
- a_t ：动作 action，发给控制器 / 底层驱动
- 一句话： $\pi_\theta : o_t \rightarrow a_t$

后面所有「格式、对齐、稀疏/稠密、H.265、清洗、标注」，本质都是：

把正确的 o_t 与 a_t 配对，或窗 / chunk，又快又对地喂进 π_θ 。

1.2 和「自动驾驶 / 游戏 AI」同一骨架，差在哪

	图像分类	游戏 RL	具身操作策略
学什么	类别	动作或价值	连续控制动作
输入	单帧图	像素/状态	多模态异构时序
监督	人工标签	奖励	多为演示动作（IL）或奖励（RL）
错了会怎样	标错类	游戏失败	撞桌、夹伤、学错相位

所以面试官问「你做数据」，他心里默认你懂：你不是在做「随便存个 bag」，而是在做 监督学习的样本工厂。

1.3 三个容易混的词（先钉死）

词	含义	常见误区
Episode	一条完整尝试（从开始到成功/失败/超时）	当成「一张图」
Step / Frame	episode 内的一个时间步	默认所有传感器同频（通常不是）
Trajectory	整段 $(o_0, a_0), \dots, (o_T, a_T)$	和「单步样本」混着说

训练时 DataLoader 抽的，往往是 某个 episode 里某个时间索引的一个训练视图（单步 / 窗 / chunk），不是整条 episode 一次塞进 GPU（除非算法特殊）。

1.4 面试开口模板

被问「机器人训练到底训什么」：

「训的是带参数的策略 π_θ ：观测进、动作出。真机主流用模仿学习，演示里的动作当 label。我后面讲的 HDF5、LeRobot、对齐，都是为了让这个监督学习的样本又对又快。」

1.5 端到端框架图（先建立全局）

图 2 · 从采集到部署：数据与训练在整条链上的位置



对应：1采集 → 2对齐 → 3归档 → 4清洗 → 5训练视图 → 6训练 π → 7部署 → 8评估 → 9回流。面试考你最多的，往往是 2~5 和 7~9；模型结构步骤 6 反而可以开源抄。

2. 用抓方块建立直觉（含多频率原始数据）

任务：7 轴机械臂 + 夹爪，抓起桌上红色方块并提起。

2.1 人怎么演示（数据从哪来）

常见采集方式：

方式	做法	数据特点
遥操作（spacemouse / VR / 主从臂）	人遥控，记指令与传感器	动作即 label，最直接
Kinesthetic teaching	手拖动机械臂	力/摩擦噪声大，要滤波
脚本 + 少量修正	固定轨迹再人工接管	分布窄，泛化差
仿真生成	规划器或 RL 产轨迹	便宜，但有 Sim2Real 沟

下面默认：遥操作 100 条成功演示（面试叙事好讲；真实项目里失败轨迹也可留作分析，是否进训练集另议）。

2.2 你「以为」数据长这样（已对齐的稠密视图）

```
episode_001（策略频率 20Hz，约 150 步 ≈ 7.5s）：
  t=0:  图像 640×480×3 | q∈R^7 | F∈R^6 → a=[+0.01,0,-0.02,0,0,0, 开]
  t=1:  图像 ...      | q ...   | F ...   → a=[+0.01,0,-0.02,...]
  ...
  t=120: ...          → a≈[0,0,0,0,0,0, 闭] ← 合爪
  t=150: ...          → a=[0,0,+0.02,...]   ← 提起

episode_002 ... episode_100
```

训练在做的事：拟合「这种画面 + 这种关节/力 → 这种动作」。

和分类比：

维度	分类	本任务策略
输入	单张图	多模态（可再加历史）
输出	猫/狗	连续向量 / 动作序列
Label	人标类名	当时下发的控制指令（或由其差分得到）
失败	错类	夹空、碰桌、相位反了

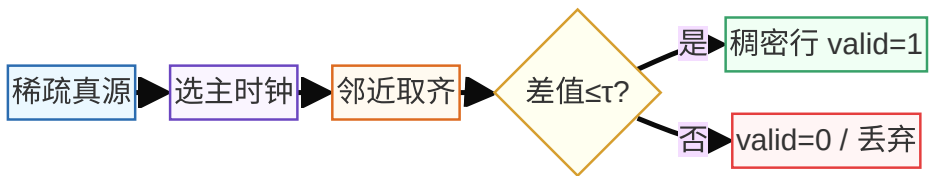
2.3 实际上盘「原始」数据长这样（稀疏、多频率）

采集端很少天生就是上表那种「一行一个完整 (o,a)」。更常见：

相机 cam_wrist @ 30Hz	→ 每帧一张图 + stamp
相机 cam_front @ 30Hz	→ 另一路图 + stamp（和腕部未必同刻）
关节状态 @ 100Hz	→ q, dq + stamp
六维力 @ 500Hz~1kHz	→ F + stamp
下发动作 @ 20Hz	→ a_cmd + stamp（策略/遥操作频率）
语言指令 @ episode 级	→ 一条字符串

同一秒内：力有几百个点，关节有一百个点，图像约 30 张，动作约 20 条。
若直接拿「最新图像 + 最新力」拼一行而不谈 stamp，相位可以偏几十到上百毫秒——策略会学到「图还没到、力已经变了」的伪相关。
这就是配套文档里 稀疏真源 → 选主时钟 → 稠密物化（容差 τ + valid） 的动机。细节与手算见配套中篇。

图 3 · 多频率对齐（为何不能「最新值直接拼」）



频率直觉：动作约 20Hz、相机约 30Hz、关节约 100Hz、力可达 1kHz。主时钟选一种写进 schema。

2.4 一条 episode 里「阶段」长什么样（建立语义直觉）

不必背，但口述时很有画面感：

阶段 A 接近：图像里方块变大；a 主要是前向/下向 delta；夹爪开
阶段 B 微调对准：位移变小；可能依赖腕部相机
阶段 C 合爪：gripper→闭；力通道出现接触尖峰
阶段 D 提起：+z；若没夹牢，后续失败（演示里应少见或标失败）

清洗时常见脏数据：合爪前后黑帧、力传感器饱和未标 invalid、动作在限位上顶死 (clip) 却当正常监督。

2.5 部署时（在线）发生什么

循环（例如 20Hz）：

- 读最新对齐观测 o_t （或与训练一致的窗）
- $\pi_\theta(o_t)$ → a_t （或先出 chunk 再按步消耗）
- 安全层：限幅 / 急停 / 与遥控仲裁
- 下发控制器
- 机器人动；传感器更新；下一拍

注意三点（面试加分）：

- 1. 训练时见过的观测分布 $\approx$ 部署时分布，否则 Sim2Real / 域偏移。
- 2. 动作语义必须一致：训练学的是 delta，部署就不能偷偷喂绝对位姿。
- 3. 延迟：若推理 80ms、控制要 20Hz，要么降频率，要么用 chunk / 异步；数据侧要记录真实 stamp 才能事后分析。

3. 输入 / 输出长什么样（张量、因果、归一化）

3.1 观测 o_t 拆开看

模态	内容	形状举例	频率直觉	对数据工程的含义
视觉	RGB（多相机）	$[3, 224, 224] \times N_{cam}$	~15~30Hz	占存储与解码大头；归档常视频，训练常缓存
深度	对齐后的 depth	$[1, H, W]$	同相机或更低	标定、空洞、单位 (m)
本体感知	关节角/速	$[DoF]$	~50~200Hz	稀疏保真；对齐时可插值（角要 wrap）
末端位姿	xyz+quat 等	$[7]$	同关节或外推	坐标系、TF、手眼标定
力/力矩	六维力	$[6]$	很高	同步容差敏感；滤波与饱和
语言	指令	token 序列	episode 级	VLA 必需；要稳定 task_id
历史	过去 k 帧	堆叠或 token 序列	—	随机访问更痛

多相机：训练时往往要求「同一决策时刻」的多视角。若两路相机 stamp 差 40ms，要在 schema 里定义：允许的 τ 、以谁为主、超差是否 valid=0。

3.2 动作 a_t：空间选择是系统设计

动作空间	含义	典型维数	优点	风险 / 代价
末端增量 delta	$\Delta xyz + \Delta rpy + gripper$	~ 7	有界、好限幅、工业常用	需约定坐标系与「每步」时间
末端绝对目标	目标位姿	~ 7	远距离运动直观	越界；与演示差分不一致时难训
关节目标 / 增量	关节空间	DoF	贴控制器	任务空间泛化要模型自己学
力矩 / 阻抗相关	力控	DoF 或任务空间	接触任务	同步、标定、安全要求高

面试加分：动作空间是 任务 + 控制器接口 的决策，不是模型「自己选的」。
落盘必须写清元数据：绝对/增量、坐标系、单位、与观测的时间关系。

3.3 监督信号的因果：三种常见约定

这是配套「对齐」章的策略侧翻译。说错因果 = 策略学歪。

约定	含义	适用
同步	a_t 是在观测 o_t 下「当时正在执行 / 应执行」的动作	部分日志字段
一步超前（常见 BC）	见 o_t ，预测马上要执行的 a_t （或 a_t 导致 o_{t+1} ）	多数模仿学习
Chunk	见 o_t ，预测 $a_{t:t+H-1}$	ACT 等

务必在文档/代码里写死一种，并在物化稠密表时遵守。不要「训练按超前，部署当同步」。

3.4 一条「训练样本」的三种视图

视图	样本内容	Dataset 压力	典型算法
单步	(o_t, a_t)	中	朴素 BC
历史窗	$(o_{t-k:t}, a_t)$	高（多次读图）	需时序上下文的策略
动作块	$(o_t, a_{t:t+H})$	高（连续 H 个 action + 边界）	ACT / 部分 Diffusion

格式选型真正要问的：你要哪种视图？shuffle 有多狠？episode 边界怎么处理（最后 H-1 步能否采样）？

3.5 归一化：常比换 backbone 更救命

连续动作若原始是「米级位移 + 弧度 + gripper $\in[0,1]$ 」混在一个向量里，MSE 会被大尺度维度主导。
常见做法：

对每个 action 维：用训练集算 mean/std 或分位数 → 训练时标准化
图像：ImageNet mean/std，或仅 /255
关节角：按关节范围线性映射到 [-1,1]

落盘时建议保留：

- 原始物理量（可复现、可换归一化）
- 或明确版本化的 norm_stats.json（与 checkpoint 绑定）

面试句：

「我们先保证 action 语义和单位正确，再做按维归一化；否则 loss 下降只是在拟合大尺度维。」

3.6 和「标注」的关系（避免串台）

概念	是不是策略的直接 label?
演示里的 a_t	☑ 是 (IL)
2D bbox / 分割	× 通常是 辅助 （检测、表示学习、质检），除非你的策略头显式吃这些
语言指令	☑ 对 VLA 是条件输入，不是「动作标签」本身

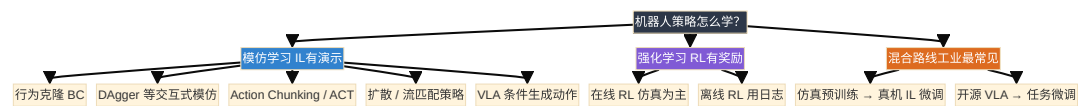
配套文档的 bbox 章，服务的是感知与质检；本篇的主监督仍是动作（或 RL 的奖励）。

4. 具体有哪些训练方法？为什么这么训？

本节回答两件事：

- 1) 业界常用的训练路线有哪些；
 - 2) 每条路线**解决什么痛点**，所以才「这么训」。
- 模型细节可开源抄；面试更爱听你能不能做方法选型。

4.0 先看总图：方法族谱



一句话选型：

你手上有什么	更该怎么训	为什么
人类演示、真机要尽快能动	模仿学习 / BC 系	真机试错贵且危险；演示直接当 label
同一观测有多种合理动作	扩散 / 流匹配，别死用 MSE	MSE 会把多峰平均成危险中间值
逐步 BC 越跑越歪	chunk / ACT，或加数据 / DAgger	复合误差；一次规划多步更稳
有语言指令、要泛化任务	VLA + 微调	用大规模预训练扛语义与迁移
仿真便宜、奖励好写	仿真 RL 预训练	样本可海量；再上真机 IL 适配
只有历史日志、不能交互	离线 RL / 保守策略优化	不能乱探索；要防外推危险动作

4.1 模仿学习族：为什么真机主流是它？

为什么先选 IL，而不是真机从头 RL？

现实约束	含义
真机慢	一条 episode 可能几十秒~几分钟，一天采不了多少
真机贵	撞坏夹爪、弄乱场景、人工重置都贵
真机危险	探索期乱出力可能伤人或设备
演示相对便宜	遥操作几十~几百条就能起步很多桌面操作

所以：真机侧默认「人类演示 → 监督学习」；RL 更多放在仿真，或做微调/精修。

方法 A：行为克隆 BC（最基础）

怎么训：数据集里每步有 o 与 a ，网络拟合 $\pi_{\theta}(o) \approx a$ ，损失常用 MSE / L1。

为什么这么训：最直接——演示者当时怎么动，模型就学怎么动。实现简单，好调试数据管线。

痛点：

- 分布偏移：推理时一旦偏出演示分布，后面更偏（compounding error）。
- 多峰动作：绕左绕右都对时，MSE 学成「中间穿过障碍」。

何时够用：任务短、场景窄、演示覆盖好；或作为基线先跑通链路。

方法 B：DAgger / 交互式模仿（治分布偏移）

怎么训：先 BC → 部署让策略跑 → 人类在策略状态上再标注正确动作 → 混进数据集再训，迭代几轮。

为什么这么训：BC 只在「人类状态」上训过；DAgger 故意在「策略自己造出来的状态」上补标签，缩小训练/部署分布差。

代价：要人反复介入，流程重；小团队未必做得起，但面试要知道「BC 失败时的正统补救之一」。

方法 C : Action Chunking / ACT (治逐步漂移 + 要平滑)

怎么训： 见 o_t，一次预测未来 H 步动作 a_{t:t+H}；损失仍是与演示 chunk 对齐（常配 Transformer）。

为什么这么训：

单步 BC：每步都可能错一点 → 误差复合 → 轨迹飘走
Chunk：短视界内按演示连贯动作走 → 少做「一步一决策」的抖与偏
再用重规划 / temporal ensemble → 把闭环找回来

何时用： 操作有明显「连贯片段」（接近、合爪、提起）；真机 BC 抖、易漂时优先试。

数据多要什么： 稠密表上能切未来 H 步；尾部要 mask/丢弃。

方法 D : Diffusion Policy / Flow Matching (治多峰)

怎么训： 不把动作当「唯一回归目标」，而当成要从噪声里生成的样本；训练学去噪（或流匹配），推理多步生成动作序列。

为什么这么训：

场景	MSE 回归	生成式策略
绕障碍可左可右	平均成撞障	采其中一种合理模式
合爪时机略有早晚	糊成含糊中间	保持多模态

代价： 算力与延迟更高；数据质量要求不降——对齐错了照样学歪。

方法 E : VLA (Vision-Language-Action)

怎么训： 大视觉-语言骨干 + 动作头；输入加语言指令；常先大规模预训练，再在你的机器人数据上微调。

为什么这么训：

- 单任务 BC 难泛化到「没见过的物体/指令」；
- 预训练带来语义与部分操作先验；
- 工业想要「一句话换任务」，必须把语言条件吃进策略。

数据多要什么： 稳定的 task/指令字段、异构数据 schema、归一化与机器人本体差异处理。

4.2 强化学习族：为什么仿真多、真机少？

在线 RL (仿真)

怎么训： 策略与环境交互；用回报 $\sum \gamma^t r_t$ 更新（PPO、SAC 等）。

为什么这么训： 没有演示时，用奖励定义「什么叫好」；仿真里可并行百万步。

真机少用冷启动的原因： 样本效率低、不安全、重置贵。常见是 仿真训完 → 真机再用 IL 微调。

离线 RL

怎么训： 只用已有日志，不再交互；要保守，避免高估未见于数据的动作。

为什么这么训： 有大量历史遥操作/自动日志，但不能让策略在真机上乱探索时用。

面试一句： 「离线 RL 不是免费的更强 BC；它在用回报信号，但必须防分布外动作。」

4.3 工业上「为什么经常混合训」



阶段	方法	为什么
仿真	RL 或大量合成演示	便宜、可重置、可随机化
真机适配	IL / 少量演示微调	对齐真实相机、摩擦、延迟
上线后	失败回流再 BC/微调	补长尾；比纯仿真猜分布靠谱

4.4 损失与「为什么用这种损失」

损失 / 目标	典型方法	为什么用它
MSE / L1	经典 BC、ACT	简单；单峰、演示一致时够用
扩散去噪损失	Diffusion Policy	表达动作分布，抗多峰
Flow matching	π_0 等	生成式另一路线，常更快采样
策略梯度 / actor-critic	在线 RL	直接优化回报，不依赖演示动作
保守 Q / 约束策略	离线 RL	防止对日志外动作盲目乐观

别背公式，要会说：

「我选损失是在选归纳偏置——单峰回归、多峰生成，还是回报最大化。」

4.5 对照表（面试速查）

方法	监督信号	主要解决啥	主要代价	真机常见度
BC	演示动作	快速有策略	分布偏移、多峰糊掉	☒☒☒☒☒
DAgger	演示 + 纠错	分布偏移	要人反复标	☒☒☒
ACT / chunk	演示 chunk	复合误差、抖动	要切 H 步	☒☒☒☒
Diffusion / Flow	演示动作分布	多峰	算力延迟	☒☒☒☒
VLA 微调	演示 + 语言	指令与泛化	数据与算力大	☒☒☒
仿真在线 RL	奖励	少演示也能学	Sim2Real	☒☒ 仿真高
离线 RL	日志 + 回报	不能交互时用回报	外推风险	☒☒

4.6 「成功标准」不是 train loss

Loss 降了只说明拟合演示。业务看：

指标	含义
真机 / 仿真任务成功率	最终目标
分阶段成功率	接近 / 合爪 / 提起哪里挂
动作饱和比例	是否总在撞限幅
干预率 / 接管率	半自动场景
p95 推理延迟	能否跟上控制频率

数据工程的指标（对齐误差、invalid 率）是 **上游健康度**，最终仍要落到成功率。

4.7 面试口述：方法选型 40 秒

「真机我们优先模仿学习，因为交互贵且危险。基线用 BC 跑通数据与部署；若轨迹抖、易漂，上 ACT 做 action chunk；若存在明显多峰绕障，换扩散或流匹配，避免 MSE 平均。要语言换任务就走 VLA 微调。仿真里可以用 RL 预训练，真机再用少量演示适配。选方法的依据是痛点：偏移、多峰、泛化、能不能交互——不是追论文名字。」

5. 训练循环细讲：从 Dataset 到 loss

5.1 表面很像训 ResNet

```
dataset = EmbodiedDataset("pick_red_cube") # 内部：LeRobot 归档 或 HDF5 热集
loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,          # ← 随机访问，视频归档的天敌
```

```
num_workers=8,
pin_memory=True,
)

model = Policy(chunk_size=10, action_dim=7, ...)
opt = AdamW(model.parameters(), lr=1e-4)

for epoch in range(num_epochs):
    for batch in loader:
        obs = batch["observation"] # 图像字典 + proprio + ...
        act = batch["action"]      # [B, H, 7] 或 [B, 7]
        mask = batch.get("valid")  # 可选：对齐失败步不进 loss

        pred = model(obs)
        loss = action_loss(pred, act, mask)
        opt.zero_grad()
        loss.backward()
        opt.step()
```

前向 / 反向 **没有魔法**。复杂度在 EmbodiedDataset.__getitem__。

5.2 __getitem__ 里实际在干什么（面试能画出来）

假设索引 i 对应「某 episode 的某时间步 t」：

- 1. 查 meta: episode_id、t、各模态文件偏移
- 2. 读图像：从 MP4 decode / 或从 HDF5|JPEG 缓存读出，resize 到 224
- 3. 读 proprio / force：从 Parquet|HDF5 按 t 取（已稠密）或现场插值（慢，一般离线物化）
- 4. 读 action：取 a[t:t+H]，处理越界
- 5. 应用归一化、可选视觉增强
- 6. 返回张量字典

每一步的坑：

步骤	坑	后果
视频随机 seek	GOP 解码放大	GPU 利用率掉到 40% 很常见
现场插值	CPU 重、还有相位错误风险	训练慢且可能学歪
chunk 越界	尾部乱拼	假 label
增强过猛	颜色/几何与标定假设冲突	真机掉点
忘记 mask invalid	对齐失败步当正样本	loss 怪、策略飘

5.3 为什么说「复杂度全在数据侧」

可抄的：
ACT / Diffusion 论文代码、LeRobot 训练入口、优化器默认值

抄不来、要你工程化的：

- 多机多相机采集同步
- 稀疏→稠密契约（主钟、τ、action 语义）
- 归档 vs 热集分层
- 清洗闸门与版本
- 部署限幅与失败回流

面试金句：

「训练 loop 和回归没本质区别；具身的差别是样本定义在时间上——错 100ms，看起来 loss 还行，真机就废。」

5.4 一个数量级直觉（口述用，精确手算见配套中篇）

假设：双相机 640×480×3，20Hz 策略频率，动作 7 维 float。

- 若存未压缩 RGB：每秒量级可到几十 MB（再乘小时 → 很快上百 GB）
- 改 H.26x 视频归档：盘省一个数量级很常见
- 但训练 shuffle 狂 seek：又逼你对 **热集** 做 HDF5/JPEG 物化

所以不是「LeRobot 和 HDF5 谁更先进」，而是 **归档省盘 vs 训练随机读** 的分层。

6. 数据格式长什么样（LeRobot / HDF5 / Parquet）

本节回答：「格式」两个字到底对应磁盘上什么文件、什么表、什么键。
选型理由见 § 7–§ 8；编码码率手算见配套文档中篇。

6.1 一张对照表（先建立坐标）

	LeRobot（常见归档）	HDF5（常见训练热集）	RLDS
本质	目录数据集：元数据 + Parquet 表 + MP4 视频	单文件（或少量分片）里的分层数组	TFRecord 轨迹生态
图像怎么存	多数：MP4（H.264 常见，H.265 可选）	多数：逐帧 JPEG/PNG 数组，或 uint8 原图	序列化进 TFRecord
状态/动作	Parquet 列（可再分稀疏/稠密）	dataset 数组 [T, D]	feature 字典
随机访问	视频 seek 贵 → 训练常另做缓存	擅长按 index 切片	偏顺序 / 框架生态
典型用途	分享、归档、省盘	DataLoader 热路径	对齐 Open X / TF

口误纠正：LeRobot ≠ 「就是 H.265」。它是 **Parquet + 视频文件** 的数据集布局；视频编码可以是 H.264/H.265 等。

6.2 端到端：同一次演示在三种「形态」里

图 4 · 同一批数据的三种落盘形态



6.3 LeRobot 目录长什么样（示意）

不同大版本细节有差异（v2 常「— episode 一堆文件」；v3 更强调 shard + meta offset）。面试画下面这张就够：

```
dataset_pick_red_cube/
├── meta/
│   ├── info.json          # fps、robot_type、特征名、维度、总 episode 数
│   ├── tasks.jsonl        # 语言指令 / task_index
│   ├── episodes.jsonl     # 每条 episode 长度、成功与否等
│   └── stats.json         # 可选：归一化用 mean/std
├── data/
│   ├── chunk-000/
│   │   ├── episode_000000.parquet
│   │   ├── episode_000001.parquet
│   │   └── ...
│   └── chunk-001/
│       └── ...
├── videos/                # 或 images/
│   ├── chunk-000/
│   │   ├── observation.images.wrist/
│   │   │   ├── episode_000000.mp4
│   │   │   └── ...
│   │   └── observation.images.front/
│   │       └── ...
│   └── ...
```

Parquet 一行（稠密训练视图示意）里常见列：

列名（示意）	类型	含义
episode_index	int	第几条演示
frame_index	int	episode 内第几步
timestamp	float	秒
observation.state	list[float]	关节等本体感知
action	list[float]	动作向量
next.done / done	bool	是否 episode 结束
图像	不进大数组时	用视频路径 + frame 索引间接引用

稀疏 vs 稠密 (同一套 Parquet 思想):

稀疏: 分表保真images/joints/forces/actions ➡ 稠密: 主钟一行img+q+F+a+valid

6.4 HDF5 里长什么样 (示意)

```
pick_red_cube_traincache.h5
├── meta/
│   ├── action_space = "ee_delta_pose+gripper"
│   ├── fps = 20
│   ├── cam_names = ["wrist","front"]
│   └── norm_stats/...
├── episode_000/
│   ├── images/wrist    dataset  shape (T, 224, 224, 3)  uint8
│   ├── images/front    dataset  shape (T, 224, 224, 3)  uint8
│   ├── proprio         dataset  shape (T, 7)           float32
│   ├── force           dataset  shape (T, 6)           float32
│   ├── action          dataset  shape (T, 7)           float32
│   └── valid            dataset  shape (T,)             uint8
├── episode_001/
│   └── ...
└── ...
```

为何训练爱 HDF5: `action[t:t+H]`、`images[t-k:t]` 是数组切片, 随机 shuffle 友好。

为何不全库永久 HDF5: 原分辨率双相机全量物化盘占用爆炸; 通常只对 **当前训练热集** 物化一版。

6.5 「格式清晰」面试口述模板

「我们归档用 LeRobot: Parquet 记状态和动作, MP4 记相机, 省盘好分享。

训练前按 20Hz 主钟做稠密对齐, 带 valid。

热集再导出 HDF5, 图像缩到 224, 方便 DataLoader 随机抽 chunk。

不是选一个万能格式, 而是 **归档一层 + 训练缓存一层。**」

7. 为什么这就倒逼这些格式

7.1 训练需求 → 数据需求 对照表

训练在要什么	数据侧必须解决	常见落点
海量图像进网络	不能无脑堆 PNG	MP4/H.26x 归档 (LeRobot 一类)
shuffle=True 抽窗/chunk	随机 seek 视频痛	热集 HDF5/JPEG 缓存
多频率传感器	相位正确	稀疏真源 + 主钟稠密
chunk / 历史	稳定索引与边界	shard、offset、episode 表 (LeRobot v3 动机)
loss 收敛	脏样本毒化	清洗 + valid
可复现实验	同一份数据可重放	schema 版本、norm 统计绑定
部署闭环	分布与安全	限幅、仲裁、失败回流

7.2 因果链（把「格式」从名词变成故事）



7.3 连回配套文档

本文问题	去配套文档看
HDF5 / LeRobot / RLDS 怎么选	上篇格式 + 下篇 Q1 + 面试官视角
视频省盘但训练慢	上篇编码速览 + 中篇码率/GOP
稀疏稠密、 τ 、action 语义	上篇对齐 + 中篇白板
清洗项	上篇清洗
bbox 等	上篇标注（辅助，不是主 π label）
ROS2 同步与安全	上篇/下篇回环

7.4 金句（说清边界）

「具身里，模型是天花板、数据是地板。架构能跟论文；**对齐、清洗、IO 分层**决定地板。我做数据工程，就是抬地板，并保证训练视图和部署观测同语义、同分布。」

8. 怎么选格式：从训练需求反推

8.1 先问四个问题（不要先报菜名）

- 1) 训练视图？ 单步 / 历史窗 / chunk? 多相机是否同刻？

2) 访问模式？ 顺序统计，还是 DataLoader 强随机？

3) 规模与协作？ 10 小时单机，还是 TB 分享与版本？

4) 生态？ PyTorch+LeRobot，还是 OpenX / TF RLDS？

8.2 决策表

场景信号	更偏向
省盘、分享、视频为主、偏顺序/按 episode	LeRobot (Parquet + MP4) 归档
训练热集、强随机、低延迟 __getitem__	HDF5 (或等价缓存) 物化
对齐开源异构 / TF 生态	RLDS 桥接
采集多频率保真	稀疏；训练前 稠密 (主钟+ τ +valid)

8.3 分层是一等公民（强烈推荐的口述结构）

- 采集/交换层：稀疏 + 视频归档 (LeRobot 等)

↓ 离线 job (可版本化、可复现)

训练层：稠密热集 (HDF5/JPEG, 按当前 π 的视图物化)

↓

DataLoader → π_θ

全库永久留「训练用原图 HDF5」往往成本爆炸；热集约一版、冷数据留归档 更常见。

8.4 反推口诀

先定 π 要的 (o,a) 视图与采样方式 → 再定归档 / 训练缓存是否分层 → 最后才谈编码参数和表字段名。

这样答「为什么选 HDF5/LeRobot」，是 从训练目标推导存储，不是背对比表。

8.5 小题：什么时候可以「先不分层」

- 数据量很小 (例如 < 几十 GB 未压缩也可接受)
- 几乎顺序读、很少随机窗
- 快速原型验证算法

面试仍建议主动说：「原型可以单层；上规模我一定拆归档和热集。」

9. 面试口述与追问

9.1 三十秒版

「训的是策略 π ：观测进、动作出。真机主流用模仿学习，因为交互贵且危险。基线 BC；抖/漂用 ACT chunk；多峰用扩散；要语言换任务用 VLA 微调；仿真可 RL 再真机 IL 适配。数据侧对齐与清洗决定策略能不能用。」

9.2 两分钟版

「以抓取为例：输入双相机 RGB + 关节 + 可选力，输出末端 delta + gripper。
方法上：先 BC 跑通；轨迹不稳就 ACT 一次出未来 H 步；绕障多解就扩散，避免 MSE 平均成撞障；要听指令再上 VLA。仿真里可 RL 预训练，真机少量演示微调。
数据上：多频率先稀疏落盘，再按策略频率稠密对齐打 valid；归档 Parquet+视频，热集 HDF5 扛随机采样。
选方法和选格式同一逻辑：先问痛点与访问模式，再选工具，不追名字。」

9.3 常见追问速查

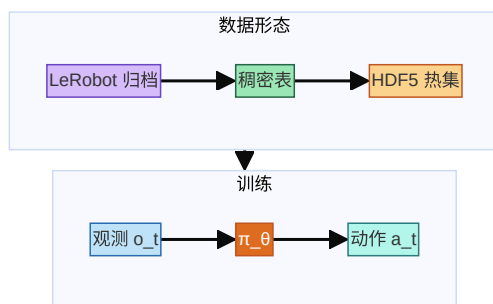
追问	怎么接
有哪些训练方法？	IL：BC / DAgger / ACT / 扩散 / VLA；RL：仿真在线、离线 RL；工业常仿真预训练+真机 IL。
为什么真机用 IL？	慢、贵、危险；演示当 label 样本效率高。
为什么上 ACT？	减复合误差与抖动；一次规划短视界动作块。
为什么上扩散？	动作多峰；MSE 会平均成不安全中间动作。
为什么不用纯 RL？	真机交互贵且危险；仿真可 RL，真机以 IL/微调为主。
delta 动作好处？	有界好限幅；贴增量接口；绝对位姿要另做越界保护。
为何 H>1？	减复合误差、更平滑；代价是 Dataset 提供未来 H 步且尾部要处理好。
为何不全库 HDF5？	原图/逐帧太贵；热集物化即可，冷数据留视频归档。
稀疏是很多零吗？	不是；是各传感器原生频率分表保真，训练再按主钟稠密。
loss 很低但真机差？	查分布偏移、因果、延迟、归一化、脏段；或方法与痛点不匹配。
和标注文档关系？	bbox 等多为辅助； π 的主监督是动作；格式/对齐保证 o 与 a 正确。

9.4 不会细节时的接法

「我这边闭环是：先固定动作空间和 (o,a) 时间语义 → 稀疏落盘 → 稠密物化带 valid → 热集缓存喂训 → 真机限幅与回流。具体编码参数/库版本我可以按你们栈对齐，但分层逻辑不变。」

10. 一张图串起来

图 5 · 总览：策略 + 数据形态



面试常考的不是「会不会写 π 」，而是会不会喂 π 。

11. 补充深挖：chunk / 扩散 / 增强 / 归一化

下面四块够面试「再问一层」；推到公式级实现仍以论文与代码为准。

11.1 ACT 的 action chunking 在干什么

问题：逐步 BC 时，若 a_t 略偏， o_{t+1} 离开演示流形，后续更偏 (compounding error)。

做法：预测整段 $a_{t:t+H-1}$ 。执行时可：

- 每步重新推理（开销大但新）
- 或执行 chunk 内若干步再重规划
- **temporal ensemble**：不同时刻预测的重叠动作加权平均 → 更稳

对 Dataset：

- 需要连续 H 个对齐好的 action
- $t > T-H$ 的样本：丢弃或 padding+mask（必须说清）
- 若 action 是 delta，chunk 内语义是「未来 H 个增量」，部署要按同一 dt 执行

口述一句：

「Chunk 用短视界开环换取更稳的动作序列，并用重规划/ensemble 把闭环找回来；数据上就是多取未来 H 步且别弄脏尾部。」

11.2 Diffusion Policy 的去噪直觉

把动作 a 看成样本。训练时：

1. 从数据取真动作 a^0
2. 加噪得到 a^k (k 为噪声强度)
3. 网络看观测 o 与 a^k ，预测噪声（或直接预测干净动作，取决于实现）

4. 损失：预测与真噪声（或真动作）的误差

推理：从纯噪声出发，多步去噪 → 得到动作。

为何适合机器人：同一 o 可能对应多种合理 a （多峰）；回归均值会糊成不安全中间动作。

数据侧：仍极度依赖对齐与清洗；扩散救不了「图和力错 200ms」的假相关。

11.3 数据增强：视觉可以凶，动作要谨慎

可做（常见）	慎做 / 别做
颜色抖动、亮度、轻微模糊	乱改与深度/力强绑定的几何，却不改 action
适度裁剪/缩放（与训练分辨率策略一致）	水平翻转（若左右约定、语言含「左/右」）
随机裁相机噪声模拟	对 action 乱加噪声当「增强」却无物理意义
域随机（仿真）	训练增强与部署预处理器不一致

原则：

改了观测里与控制相关的几何，就要同步改 label 或干脆别改。

11.4 归一化与 loss 再多说一点

- 按维标准化 后，MSE 才大致「各维平等」。
- gripper 常已是 $[0,1]$ ，与米级 Δx 混用必炸。
- 换数据集或夹爪行程后，**必须重算或明确沿用哪版 stats**，并与权重绑定。
- 评估时看物理单位下的误差（mm、deg），不要只看标准化空间 loss。

和配套文档怎么配合读

你的状态	先读	再读
不懂「为啥整天讲格式」	本文 §1–§2、§6–§7	配套「面试官视角」+ 上篇格式
要看目录树 / 表结构	本文 §6	配套上篇细讲
要啃选型/坑/手算	本文 §8 当索引	配套上篇 + 中篇 + 下篇
要懂算法对数据的额外要求	本文 §4、§11	论文/开源代码
临场两分钟	本文 §9	配套下篇对应题

自检清单（读完能否口头答出）

- ☐ 用一句话说清 π 的输入输出
- ☐ 解释为何原始数据是多频率、训练却要稠密行
- ☐ 说出至少一种 action 因果约定

- ☐ 对比 IL / RL 各适合哪
- ☐ 说明 ACT chunk 对 Dataset 多要什么
- ☐ 能画出图4/图5，并口述 LeRobot 目录与 HDF5 键
- ☐ 用训练需求四个问题做一次格式选型口述

全部能答，再去啃配套文档的细节题，不会晕在名词里。